Local Info only

Lorenzo Pecorari - 1885161

Lighten the observation

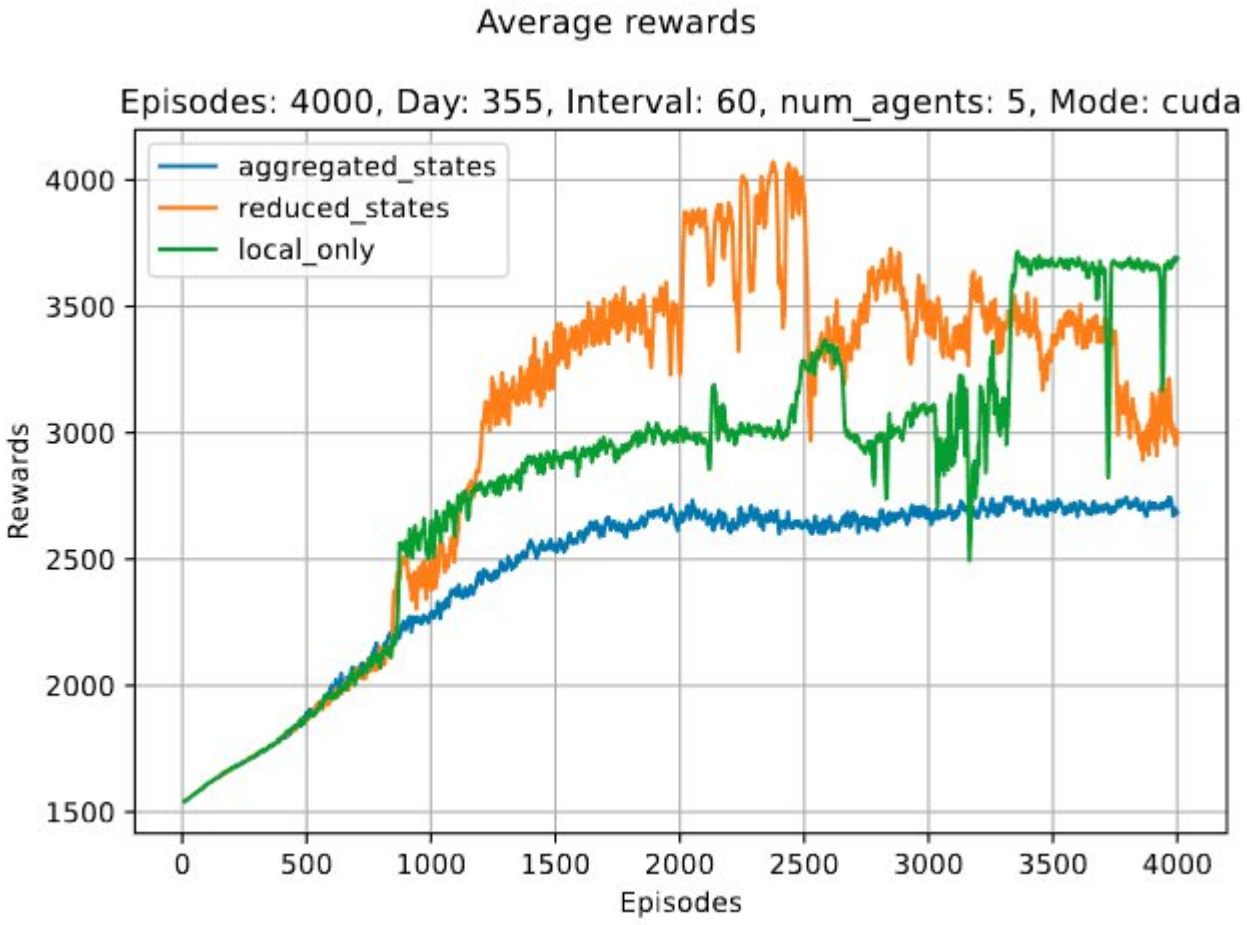
- From $s_t^i = [b_t^i, q_t^i, \tau_t^i, mb_t^i, Mb_t^i, \mu b_t^i, mq_t^i, Mq_t^i, \mu q_t^i]$
- To $s_t^i = [b_t^i, q_t^i, \tau_t^i]$
- Agents can rely only on information coming from their local state
- Observation with <u>same dimensionality</u> of single agent formulation
- Smaller observation space wrt initial formulation
- Interesting to compare performances between “full” aggregated, partial and no global information

Comparisons

- "**Local only**" $\rightarrow$ $s_t^i = [b_t^i, q_t^i, \tau_t^i]$
  - Agent only knows its battery, backlog and time
- "**Reduced states**" $\rightarrow$ $s_t^i = [b_t^i, q_t^i, \tau_t^i, \mu b_t^i, \mu q_t^i]$
  - Agent knows also average battery and average backlog status of others
- "**Aggregated states**" $\rightarrow$ $s_t^i = [b_t^i, q_t^i, \tau_t^i, mb_t^i, Mb_t^i, \mu b_t^i, mq_t^i, Mq_t^i, \mu q_t^i]$
  - Agent knows also min and max of battery and backlog status of others

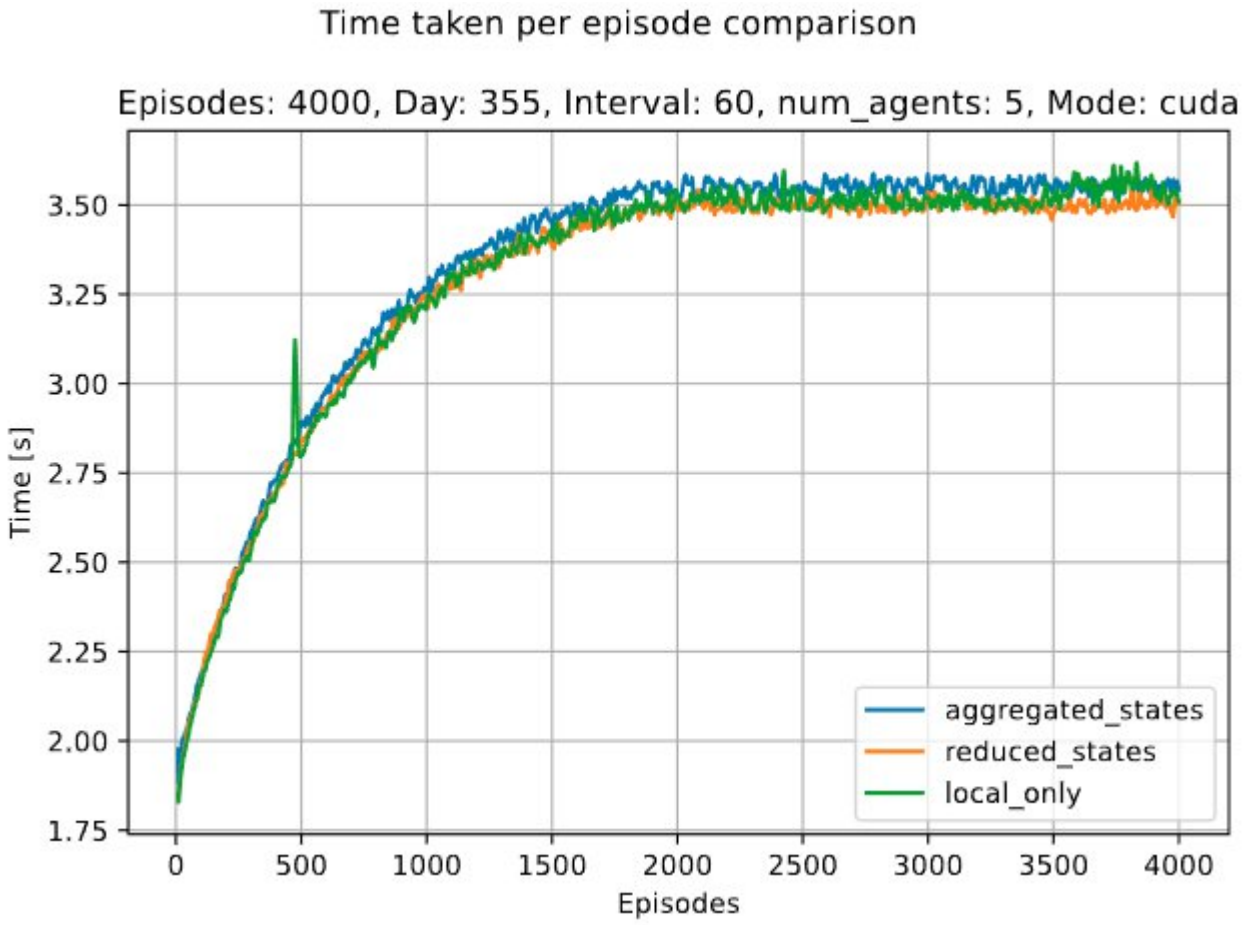
Rewards comparison

- State aggregation leads to lower rewards with a more stable curve
- Giving only local information to agents determines better rewards than aggregated info
- Allowing agents to observe an average of others' metrics seems to be the better trade-off between the configurations



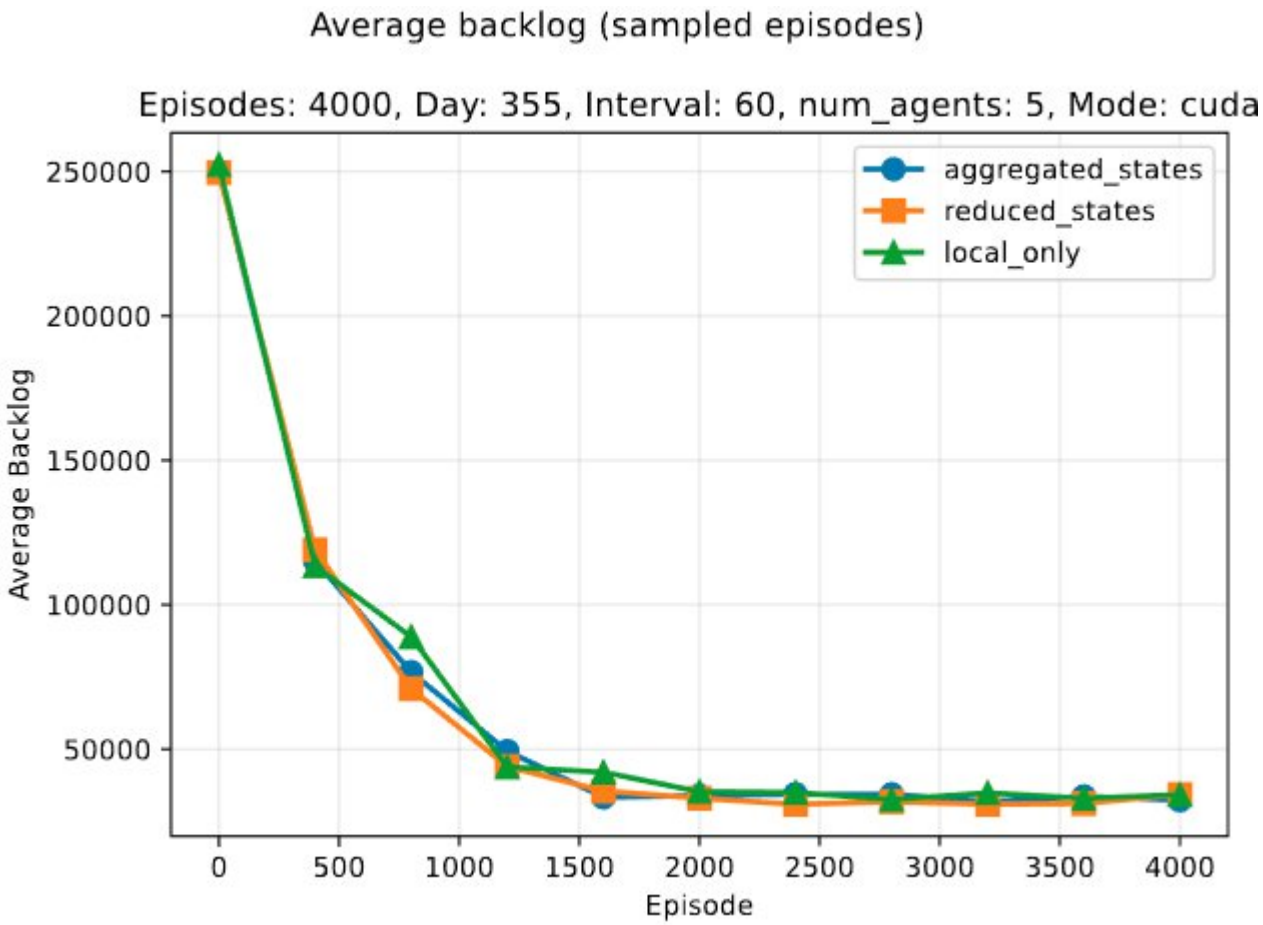
Time comparison

- Temporal performances seems to be very similar across the different configurations
- No relevant differences between “heavier” informations



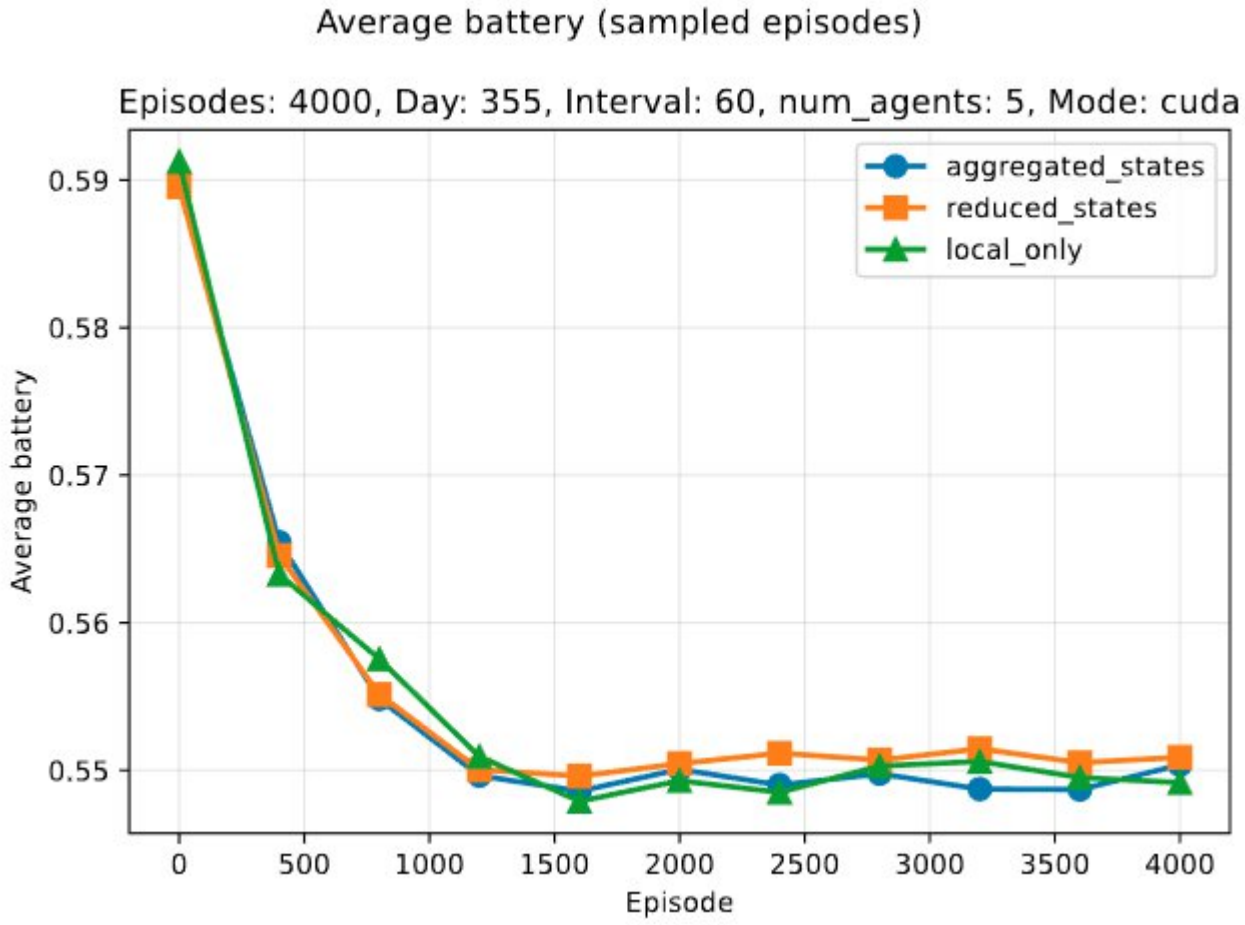
Backlog comparison

- No significant differences between different configurations
- “Reduced states” slightly better than others two



Battery comparison

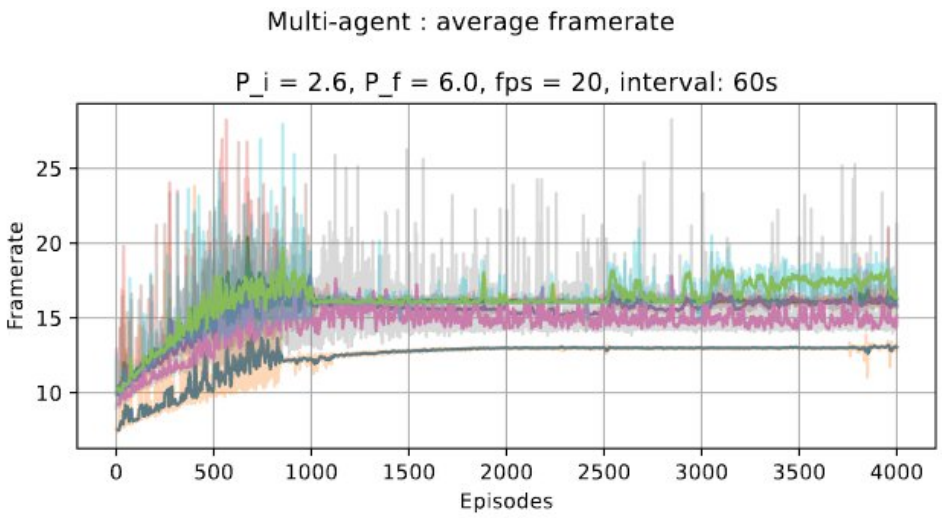
- Better battery management for “reduced states” configuration
- “aggregated states” tends to consume more energy than others



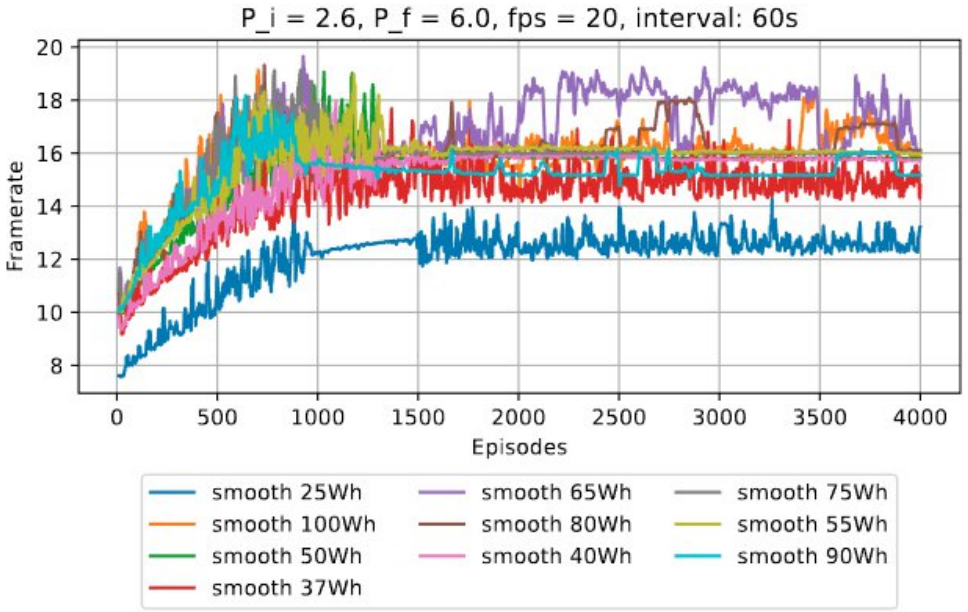
Framerate comparison

- “Reduced states” configuration shows faster convergence and more stable if compared with others

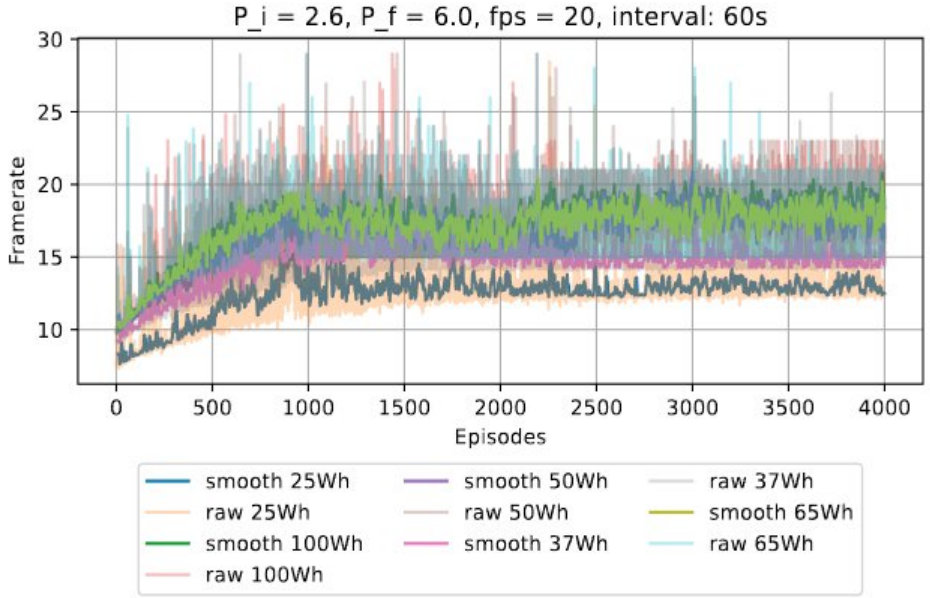
REDUCED STATES



LOCAL INFO ONLY



AGGREGATED STATES



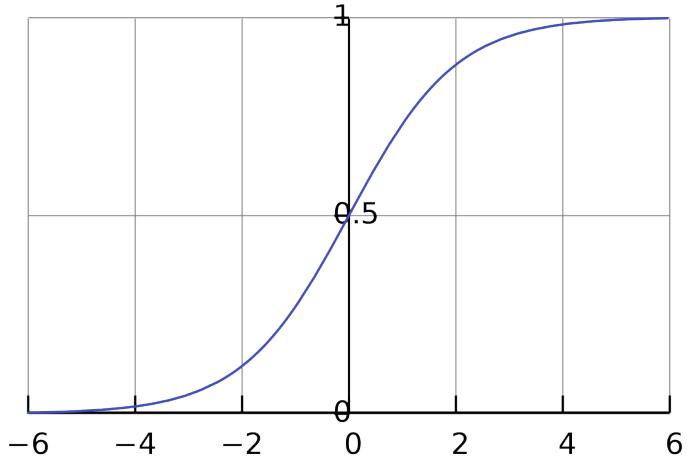
More expressive information

Z score and sigmoid

Limiting the z-score

- Due to the range (-∞, +∞) of the output of z(x), it can be applied a limitation of its value to smaller intervals
- Ex: sigmoid function and sigmoid can cope such problem

- sigmoid: $\mathbb{R}^k \rightarrow [0, +1]$
- $\text{sigmoid}(x) = \sigma(z(x)) = \frac{1}{1+e^{-x}}$



- Agent in:
  - Similar conditions → ~0.5
  - Better conditions → >0.5
  - Worse conditions → <0.5

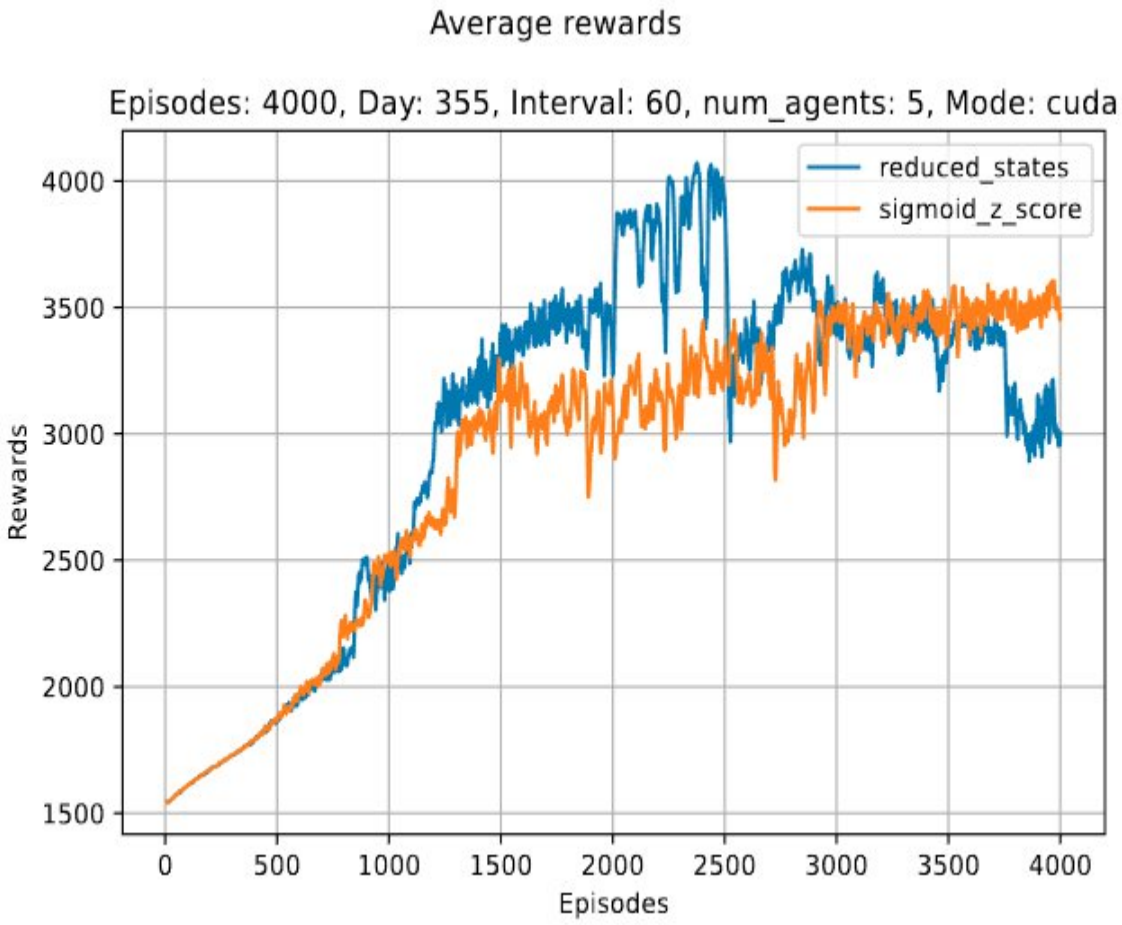
Formulation with sigmoid

- Using z-score for finding position of agent metrics wrt the group
- Using tanh for delimit the z-score
- From $s_t^i = [b_t^i, q_t^i, \tau_t^i, mb_t^i, Mb_t^i, \mu b_t^i, mq_t^i, Mq_t^i, \mu q_t^i]$
- To $s_t^i = [b_t^i, q_t^i, \tau_t^i, \sigma(z(b_t^i)), \sigma(z(q_t^i))]$
- where:
  - $\sigma(z(x)) = \frac{1}{1 + e^{-x}}$

from 9 dimensions to 5

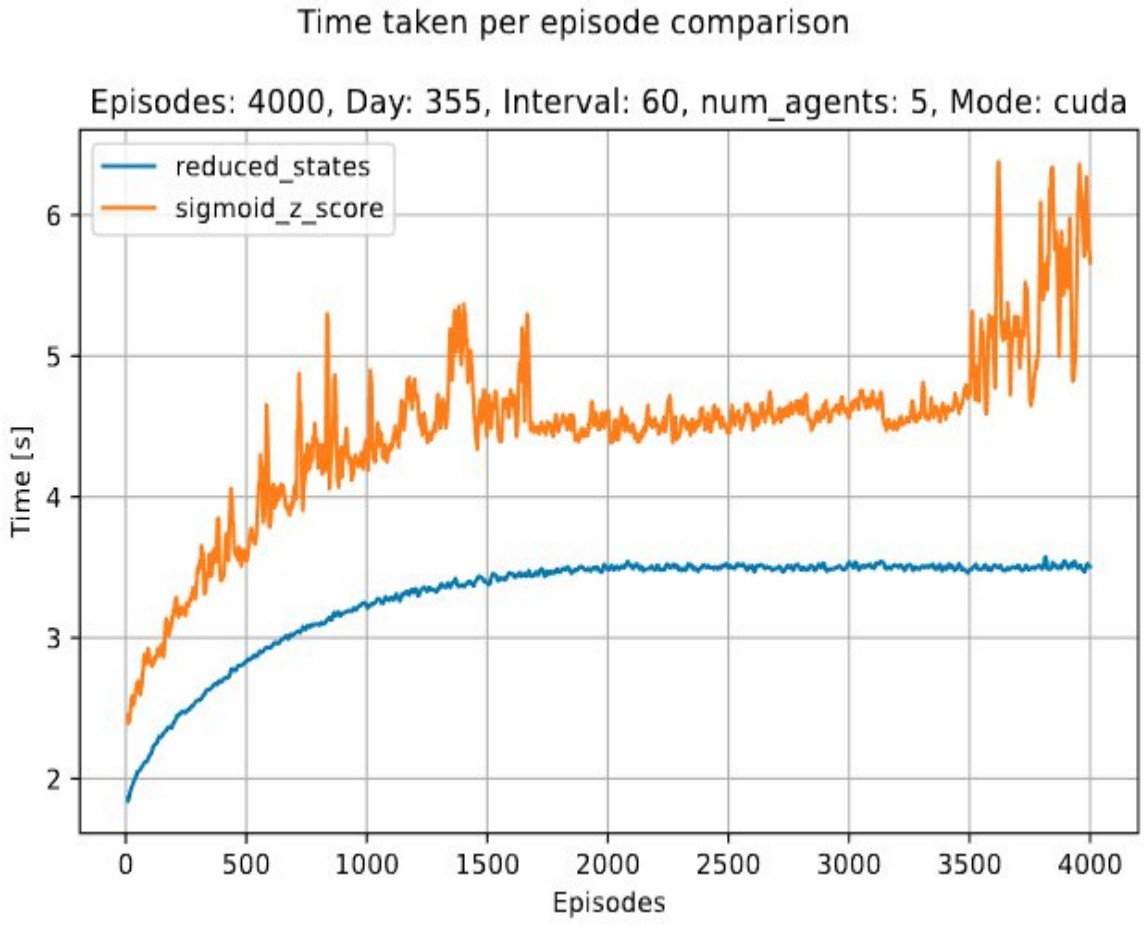
Rewards comparison

- Adoption of sigmoid and z-score introduces more gradual growth of rewards with overtake in final phases of training



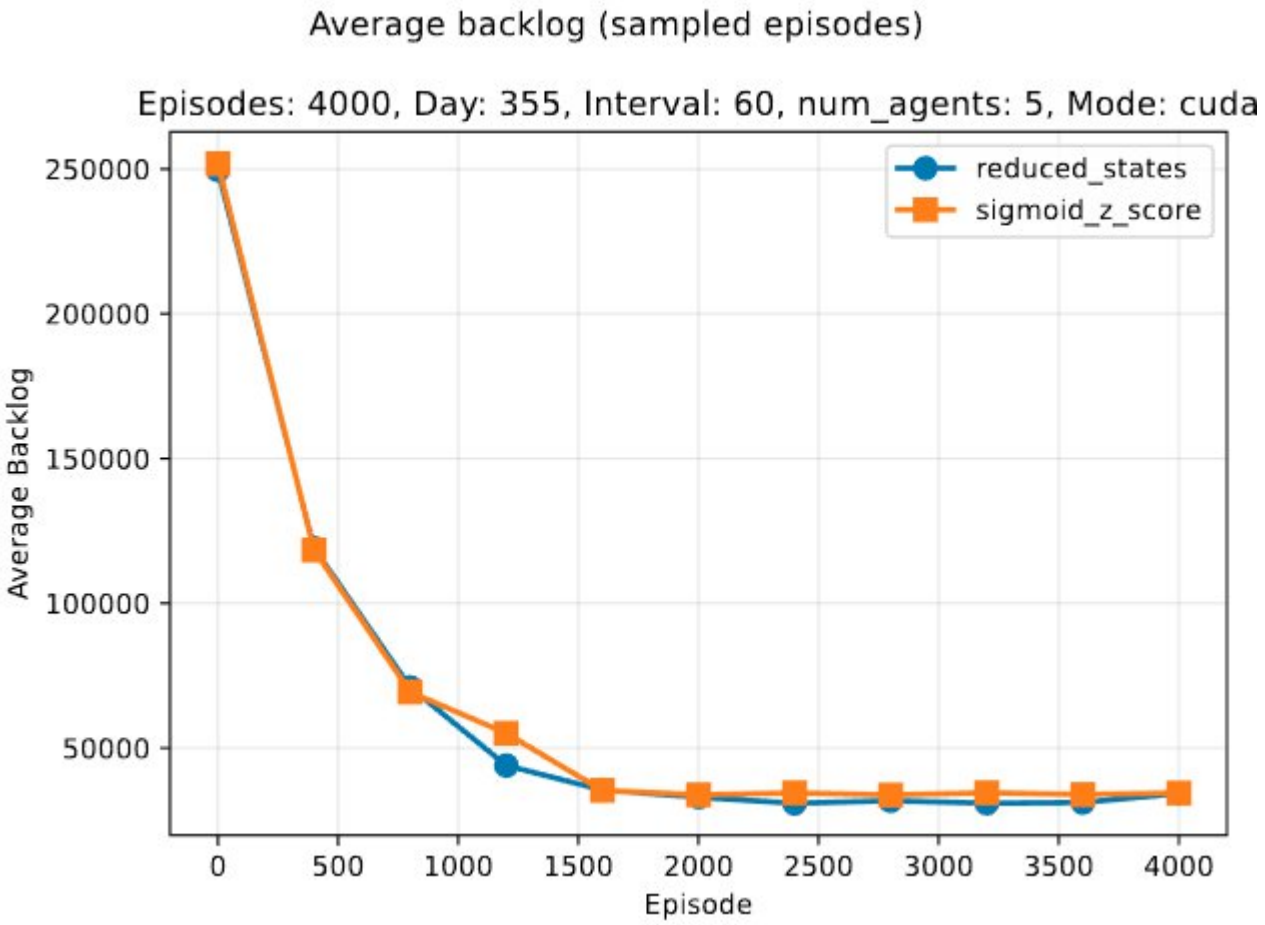
Time comparison

- Higher time required by newer configuration
- Note: reduced states uses library function “numpy.mean”, which is optimized while sigmoid configuration uses a hand-made version



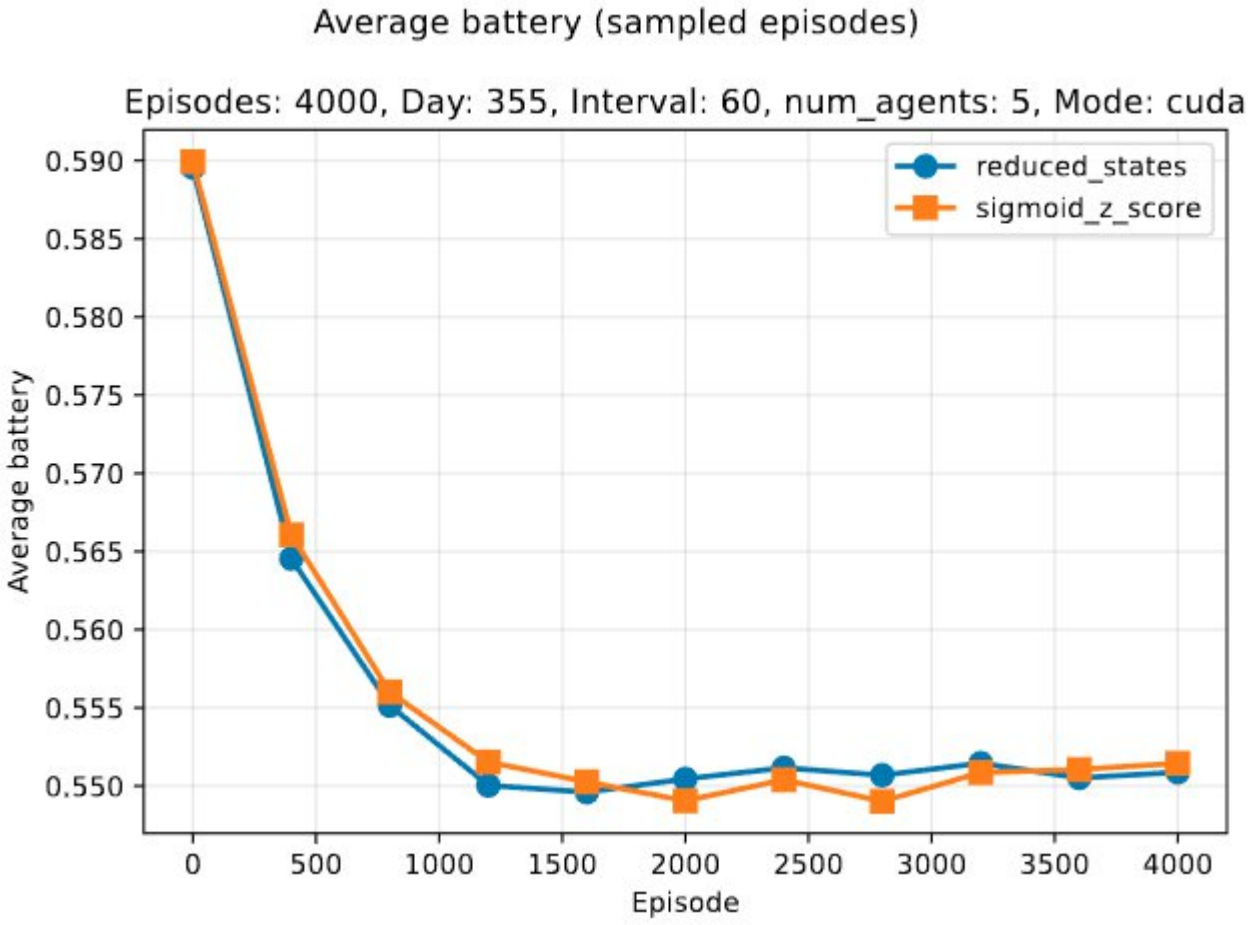
Backlog comparison

- Slightly higher average backlog for configuration using sigmoid and z-score
- Convergence to same result after epsilon reaches the min value



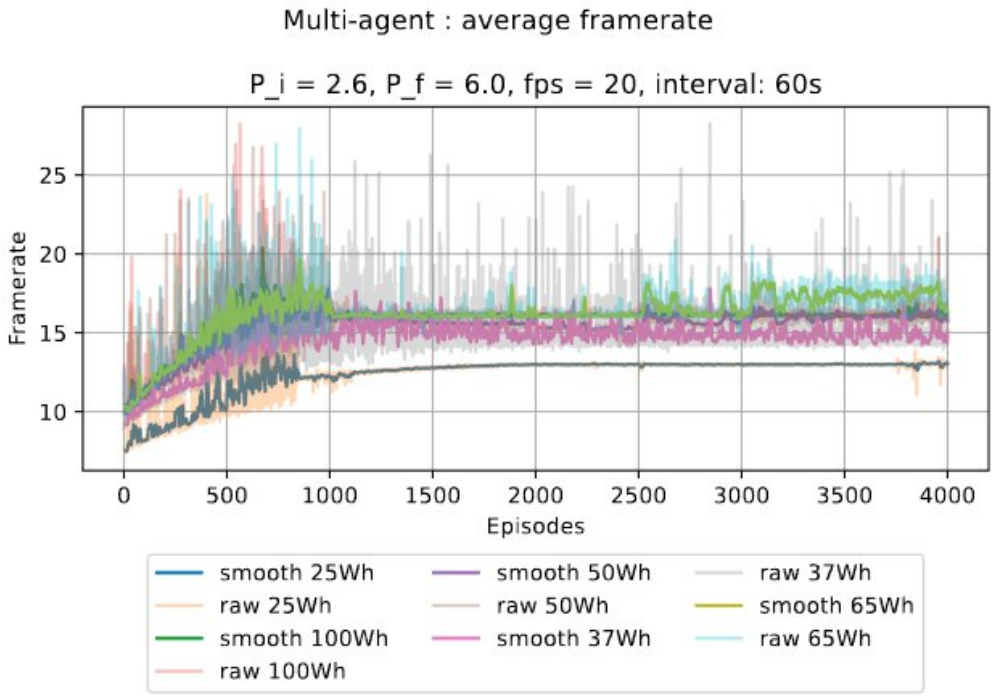
Battery comparison

- Initial advantage of sigmoid configuration with interleaving along the training
- Slightly advantage in final phases of episodes for newer configuration

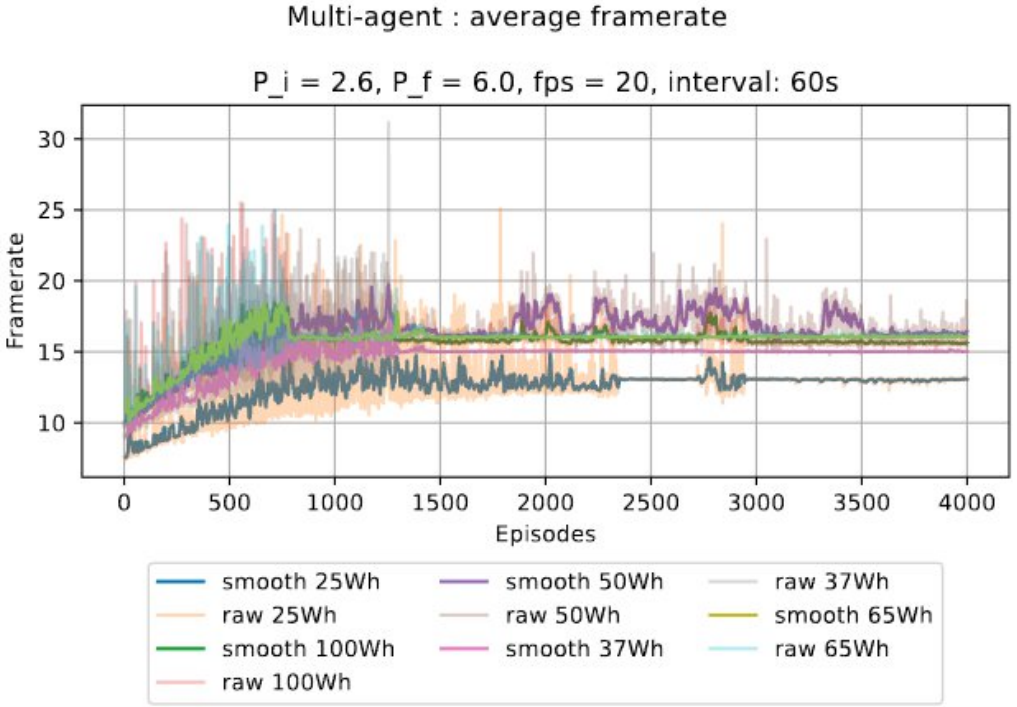


Framerate comparison

REDUCED STATES



SIGMOID Z SCORE



- No relevant differences between the configuration
- Slightl greater oscillation of second version